

Optimizing Source Locations in Clang Modules

Recycling allocations to keep the 32-bit address space alive

Ayokunle Amodu

University of Alberta

Mentor: Vassil Vassilev · Princeton / CERN
CppAlliance · Compiler Research Group

D U R A T I O N

May 4 – August 31, 2026

Where we're going

A bottom-up tour of one specific corner of Clang

01

Background

Textual headers, modules, the serialization pipeline

02

Source Locations

What a SourceLocation actually is, and the table behind every error

03

The Problem

Why a typical libc++ build sits 29,897 bytes from the ceiling

04

The Fix

Recycle allocations: DenseMap prototype → IntervalMap upgrade

05

Plan & Validation

Timeline, risks, expected numbers

Textual headers: copy-paste compilation

main.cpp

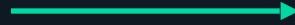
```
#include "math.h"

int main() {
    return add(1, 2);
}
```

math.h

```
int add(int a, int b);
```

preprocessor



(textual
substitution)

main.cpp (after preprocessing)

```
// from math.h
int add(int a, int b);

int main() {
    return add(1, 2);
}
```

The header isn't "imported." Its text is inlined into every translation unit that includes it.

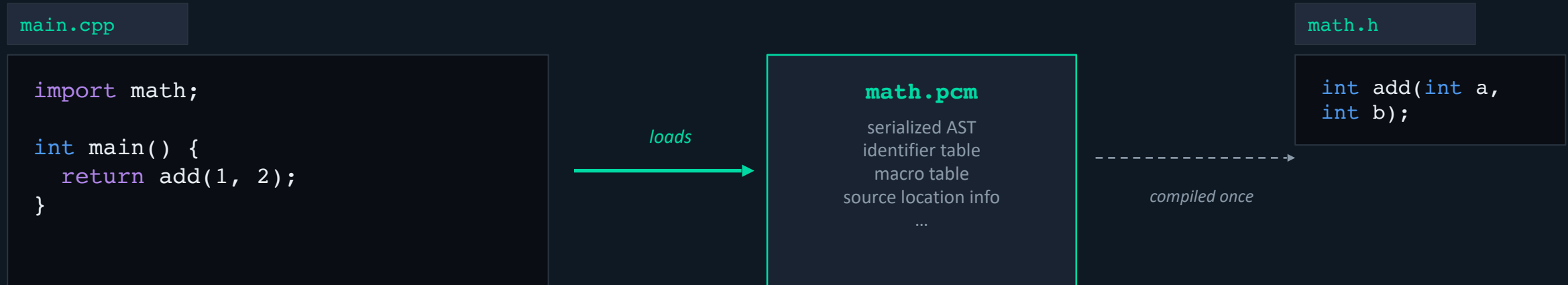
→ the compiler re-parses `math.h` every time it's seen.

...and the parser pays the toll on every TU



3× parsed. **3×** tokenized. **3×** AST built. *Same file, every time.*

Modules: parse once, import many



What changes

Textual #include

Header inlined into every TU

Re-parsed N times

Source locations local to each TU

Module import

AST shared across TUs

Parsed once into .pcm

Source locations carried inside the .pcm

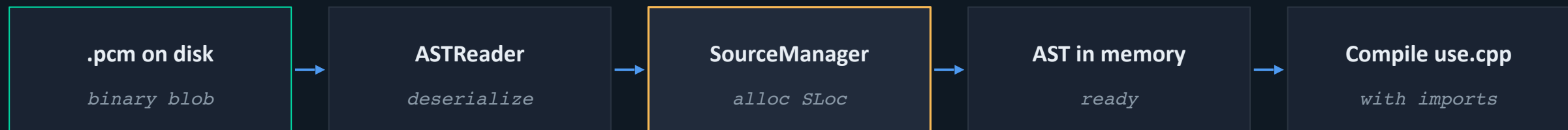
The module serialization pipeline

WRITE → on disk



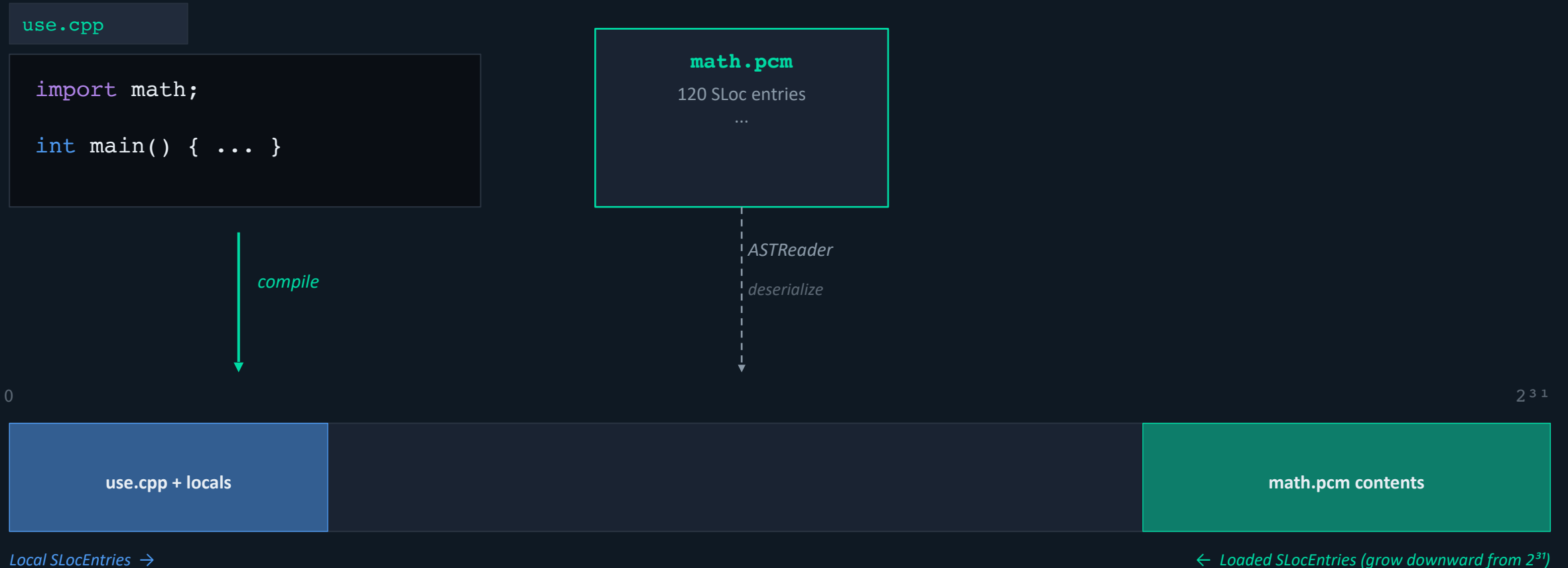
----- D I S K -----

READ ← back into compiler



We're focused on what happens **inside SourceManager** when ASTReader deserializes a module.

Loading a module fills the address space



Every import puts data somewhere in this 32-bit number line. The question becomes: **what does "somewhere" actually mean?**

 clang++ -fmodules use.cpp

```
$ clang++ -fmodules -fimplicit-module-maps use.cpp
```

```
math.h:3:5: error: expected ';' after top level declarator
```

```
3 | int add(int a, int b)
  |      ^
```

How does Clang know it's

math.h : **3** : **5** ?

A SourceLocation is just a 32-bit offset

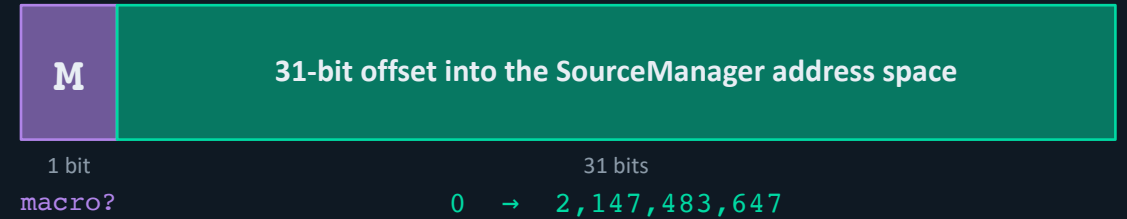
clang/Basic/SourceLocation.h

```
class SourceLocation {
public:
    using UIntTy = uint32_t;

private:
    UIntTy ID = 0;

    enum : UIntTy {
        MacroIDBit = 1U << 31
    };
};
```

32-bit layout



math.h:3:5

is not stored as ("math.h", 3, 5)

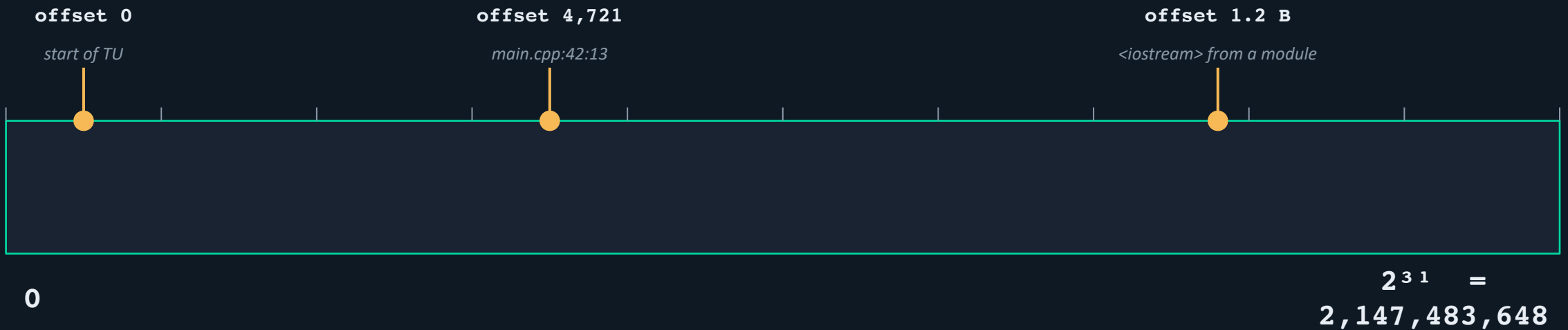
It's stored as **a single number**

Why so tight? Cache locality. SourceLocation lives inside common AST nodes like DeclRefExpr — millions of them per build.

One global number line

Every byte of source — `main.cpp`, `math.h`, every macro expansion — lives somewhere on this line.

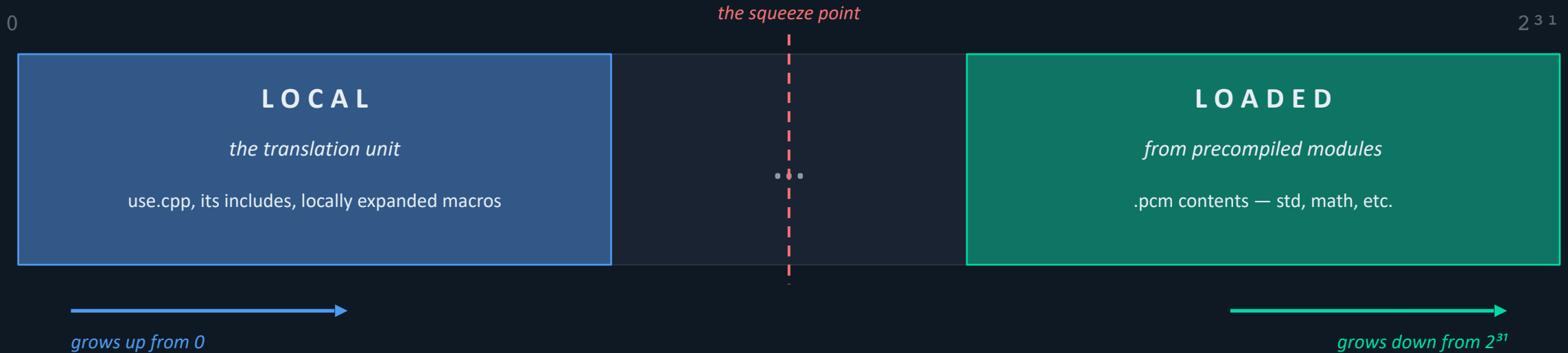
31-bit address space · one per compilation



A single number identifies a location uniquely. But how does **4,721** know which file it belongs to?

→ The address space is divided into **ranges**, each owned by an **SLocEntry**.

Two regions, growing toward each other



Local entries grow up from 0. **Loaded entries** grow down from 2^{31} .

When they collide, **the build dies**.

Today we're worried about **the loaded side** — that's where modules live.

What lives at each offset: SLocEntry

A discriminated union — either a piece of a file, or a macro expansion.

clang/Basic/SourceManager.h (simplified)

```
class SLocEntry {
    unsigned Offset : 31;    // position
    unsigned IsExpansion : 1;
    union {
        FileInfo File;      // a piece of source file
        ExpansionInfo Expansion; // a macro expansion
    };
};
```

Two flavors

FileInfo

- a contiguous slab covering one file
- knows the FileEntry, include location, contents

ExpansionInfo

- a macro's expansion site
- knows the spelling loc and expansion range

Key insight. Each **SLocEntry** owns a **range** of the 31-bit address space. The *next* entry starts where this one ends.

→ Putting them all in order gives us... a **table**.

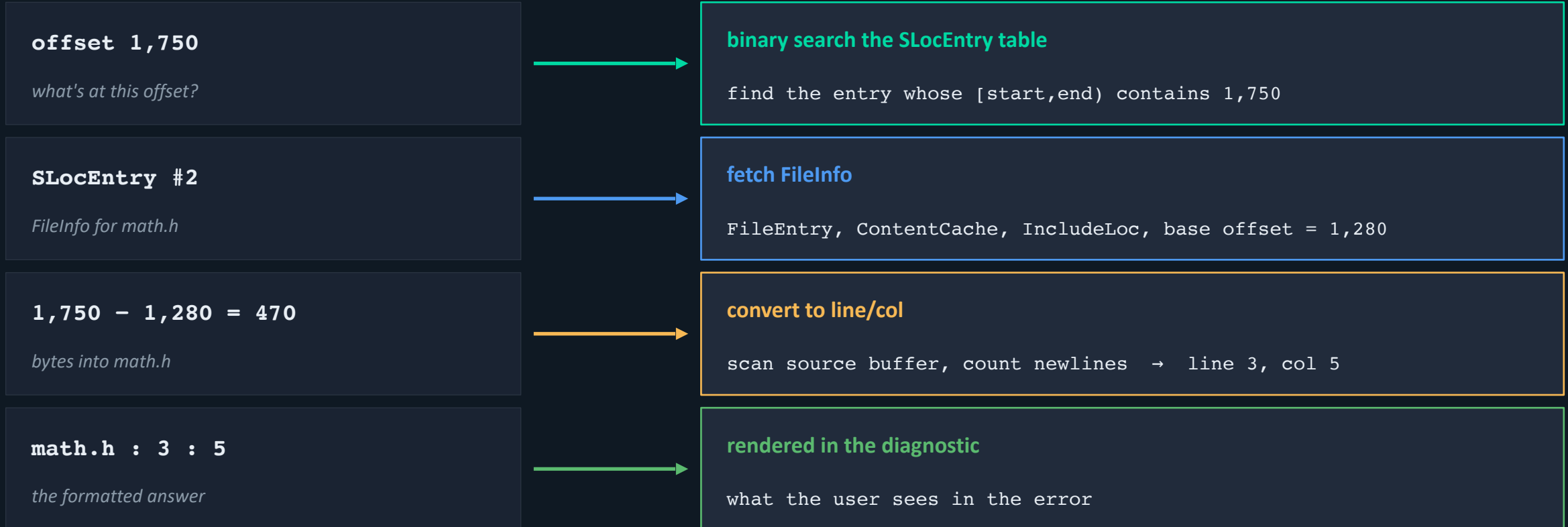
The SLocEntry table

A sorted vector of (Offset, SLocEntry) pairs. Binary search by offset → SLocEntry → file/line/col.

#	Kind	File / Macro	Offset range	Size	What lives here
0	File	main.cpp	[0 .. 1,250)	1,250	user's TU
1	File	<built-in>	[1,250 .. 1,280)	30	compiler builtins
2	File	math.h	[1,280 .. 5,001)	3,721	included by main.cpp
3	Macro	ADD(x,y)	[5,001 .. 5,022)	21	macro expansion site
4	File	iostream (from std.pcm)	[2 ³¹ -500K .. 2 ³¹ -400K)	100K	loaded module
5	File	vector (from std.pcm)	[2 ³¹ -400K .. 2 ³¹ -300K)	100K	loaded module

Given offset **1,750**, binary search finds row #2 → **math.h**, then $1,750 - 1,280 = 470$ bytes into the file → **line 3, col 5**.

SourceManager: the oracle that turns offsets into locations



One `uint32_t` carries an enormous amount of meaning — but only because of the table behind it.

Putting it together: from offset to the error you saw

```
math.h:3:5: error: expected ';' after top level declarator
```

1. SourceLocation

stored in the AST

```
// Parser hit an error
SourceLocation Loc;

Loc.getRawEncoding()
// → 1,750
```

2. SLocEntry

look up in table

```
SourceMgr
  .getSLocEntry(Loc);

// → FileInfo for
//   math.h
```

3. FileInfo

subtract base offset

```
auto base = 1280;
auto delta = 1750 -
1280;

// → 470 bytes
//   into math.h
```

4. Line / Col

scan the buffer

```
scanNewlines(
  buffer, 470);

// → line 3
//   col 5
```

Everything **the user sees** — the file name, the line, the caret position, the squiggly underline — is reconstructed on demand from **a single 32-bit integer** and the **SLocEntry table**.

When the table runs out of room ... so does the compiler.

An experiment in duplication

Four files. Two modules. One header included by both.

a.h

```
#ifndef A_H
#define A_H
#include <iostream>
int mod_A = 42;
#endif // A_H
```

b.h

```
#ifndef B_H
#define B_H
#include <iostream>
int mod_B = 21;
#endif // B_H
```

use.cpp

```
#include "a.h"
#ifdef TWO
    #include "b.h"
#endif
int main() { /* use mod_A / mod_B */ }
```

module.modulemap

```
module A {
    header "a.h"    export *
}
module B {
    header "b.h"    export *
}
```

← both modules pull in <iostream>

Step 1. Compile without -DTWO · just module A loads

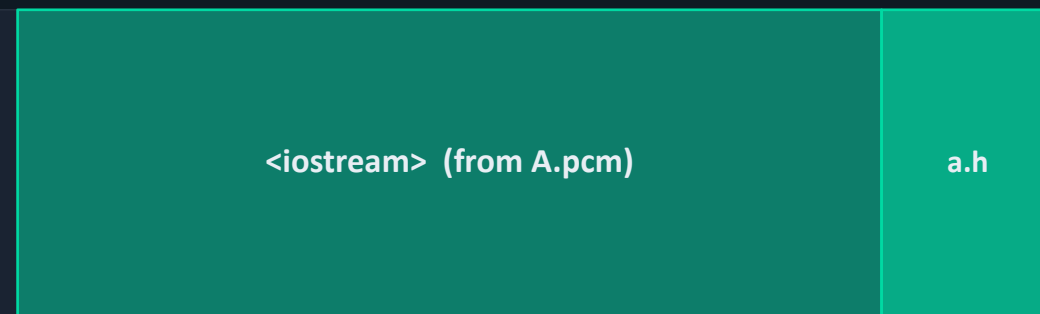
```
$ clang++ -fmodules -fimplicit-module-maps -fmodules-cache-path=. use.cpp -Xclang -print-stats
```

Local



0

Loaded (A.pcm)

2³¹

AST File Statistics

5,827

source location entries (denominator)

from -print-stats

*** AST File Statistics:

```
278/5827 source location entries read (4.77%)  
20/30960 types read  
125/29037 declarations read  
1485/5199 identifiers read
```

Step 2. Add -DTWO · now both modules load

```
$ clang++ -DTWO -fmodules -fimplicit-module-maps -fmodules-cache-path=. use.cpp -Xclang -print-stats
```

Local

Loaded — every module gets its own copy



AST File Statistics

11,650

source location entries

exactly 2× the one-module baseline

*** AST File Statistics:

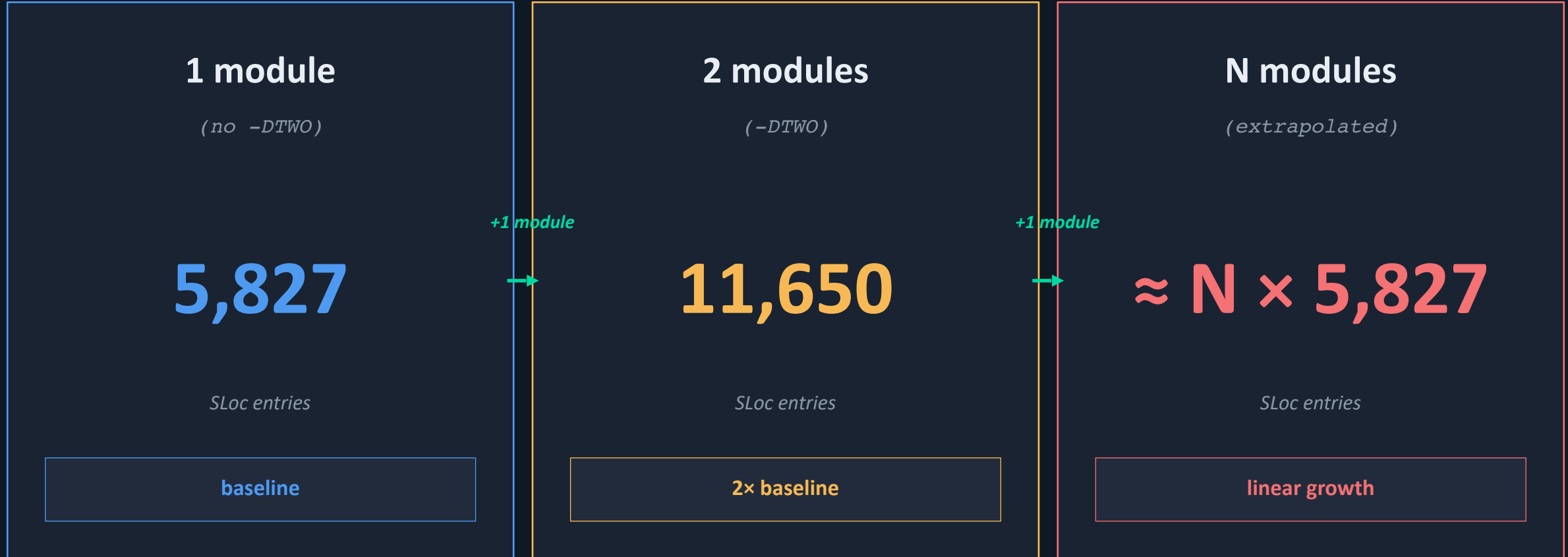
574/11650 source location entries read (4.92%)

37/61916 types read ← also 2×

245/58068 declarations read ← 2×

2965/10386 identifiers read ← 2×

It's linear in module count, not unique files



Every additional module that pulls in `<iostream>` allocates a **fresh range** for the same file. The address space scales with **module count**, not unique content.

Real-world: a libc++ build sits 29,897 bytes from the ceiling

Importing `std.pcm`, `std_core.pcm`, and `Darwin.pcm` — for a program that just calls `std::cout` once.

29,897 bytes left

ALLOCATED · `std` + `std_core` + `Darwin` · 2,147,453,751 bytes

0

$2^{31} = 2,147,483,648$

96 MB

consumed by the standard library alone

2,147,453,751

last allocated offset (out of 2,147,483,648)

29,897 B

room remaining before clang aborts

Each additional module import eats into that 29,897-byte margin. And that space doesn't recover between translation units.

→ *The real-world problem is not theoretical.*

Why not just widen SourceLocation to 64 bits?

LLVM Discourse · 2025

Revisiting 64-bit Source Locations

Proposed widening UIntTy to uint64_t.

LLVM Discourse · 2025

RFC: Opt-in CMake Option for 64-bit SourceLocation

Proposed making 64 bits a build-time toggle.

NEITHER THREAD CONVERGED ON
ADOPTION

THE OBJECTION

SourceLocation is embedded inside common AST nodes — **DeclRefExpr**, **DeclaratorDecl**, every **Token**, every **Stmt**. Millions of instances per build. Widening adds **+4 bytes everywhere** — for every user, not just those near the ceiling.

The community's takeaway: *don't pay a blanket cost for a localized problem.*

→ explore **recycling strategies** first. *That's exactly what this work does.*

Our beloved allocator

Five lines. Called once per module. Allocates unconditionally — no check, no memory.

clang/Basic/SourceManager.cpp

```
std::pair<int, SourceLocation::UIntTy>
SourceManager::AllocateLoadedSLocEntries(
    unsigned NumSLocEntries,
    SourceLocation::UIntTy TotalSize) {

    LoadedSLocEntryTable.resize(LoadedSLocEntryTable.size() + NumSLocEntries);
    NextLoadedOffset -= TotalSize;

    return {-(int)LoadedSLocEntryTable.size(), NextLoadedOffset};
}
```

Line 1 of body

Reserve N more SLocEntry slots *regardless of whether identical files have already been seen.*

Line 2 of body

Advance the watermark down *even if every byte of TotalSize is for a file we've allocated five times already.*

...and where it's called from

clang/Serialization/ASTReader.cpp

```
// in ASTReader::ReadASTBlock(), one per loaded module file F:
case SOURCE_LOCATION_OFFSETS: {
    // F.LocalNumSLocEntries comes from the .pcm header
    // SLocSpaceSize is computed from the SLoc offset table

    std::tie(F.SLocEntryBaseID, F.SLocEntryBaseOffset) =
        SourceMgr.AllocateLoadedSLocEntries(F.LocalNumSLocEntries, SLocSpaceSize);

    // ...record (BaseID, BaseOffset) so this module's
    // local SLoc offsets can be translated to global ones
    break;
}
```

Per module → one call → one new region.

*ASTReader does not ask whether F's input files have already been allocated by an earlier module. It doesn't have to — **no one's keeping that record.***

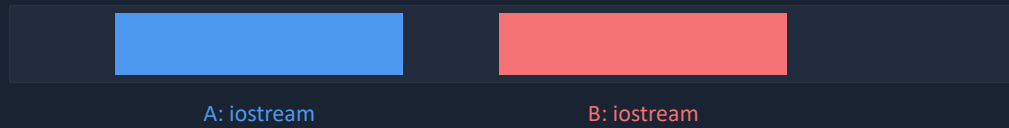
What if we just... checked first?

BEFORE

Today's behavior

Module A loads `iostream` → allocate [`X` .. `X+N`]

Module B loads `iostream` → allocate [`Y` .. `Y+N`] (*dup*)



Two slabs. Identical bytes. 31-bit space spent twice.

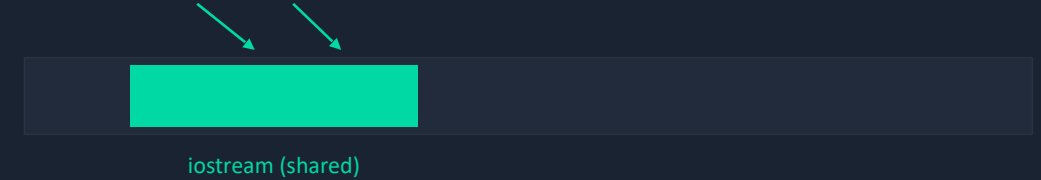
cost grows with module count

AFTER

Recycle the slab

`iostream` allocated **once** at [`X` .. `X+N`]

Module A & B both **rewired** to reuse it

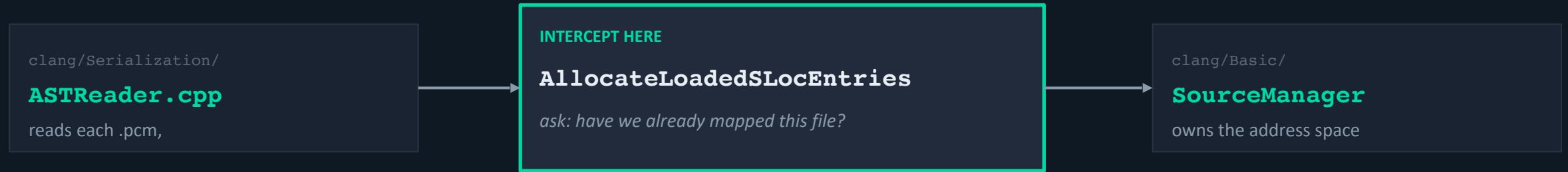


One slab per unique file. Identity ≠ count.

cost grows with unique input files

Same address space. Same SLocEntries. **We just remember what we've already given out.**

One place to intervene



Today

```
// every module → fresh region
auto [id, off] = Allocate(N, size);
```

With dedup

```
// pass file identity → maybe reuse
auto [id, off] = Allocate(N, size, F);
```

Why this point and not the call site?

Because `AllocateLoadedSLocEntries` is the **single place** in Clang that hands out loaded address space. Every reader — `ASTReader`, `PCH`, future module formats — **funnels through here**. Fix it once, fix it everywhere.

First, a DenseMap to prove the idea

clang/Basic/SourceManager.h

```
// new state on SourceManager:
DenseMap<FileEntryRef,
         std::pair<int, SourceLocation::UIntTy>>
    LoadedFileEntryAllocations;
```

the shape of the table

```
key   FileEntryRef
      the file's identity

val   (BaseID, BaseOffset)
      where it already lives
```

Inside AllocateLoadedSLocEntries:

```
if (auto it = LoadedFileEntryAllocations.find(F); it != end()) {
    ++NumRedirectedAllocations;           // -print-stats counter
    return it->second;                     // reuse existing slab
}
// fall through to existing allocation, then record:
LoadedFileEntryAllocations[F] = {BaseID, BaseOffset};
```

$O(1)$ lookup. Identical files map to identical slabs. *If the numbers don't move, identity isn't reaching here yet.*

Where the change actually lives

1

SourceManager.h

Declare the map

```
class SourceManager {
    ...
    DenseMap<FileEntryRef,
        std::pair<int,
            UIntTy>>
        LoadedFileEntry
        Allocations;
};
```

Add the DenseMap field. New SourceManager state — that's it.

2

SourceManager.cpp

Lookup before alloc

```
Allocate(N, size, F) {
    if (F && map.find(F))
        return hit;    //reuse
    ...existing alloc...
    // record on miss
    if (F)
        map[F] = {id, off}
}
```

In AllocateLoadedSLocEntries, check the map; reuse on hit, record on miss.

3

ASTReader.cpp

Pass file identity

```
case SOURCE_LOCATION_
OFFSETS: {
    // identity for F
    FileEntryRef Fid =
        F.getInputFile();
    std::tie(...) =
        SourceMgr.Allocate(
            N, size, Fid);
}
```

In the SOURCE_LOCATION_OFFSETS case, hand the FileEntryRef of F down through the call.

Three files. No new data structures outside SourceManager. **All other code keeps speaking the same (BaseID, BaseOffset) pair.**

Then, an IntervalMap for the production fix

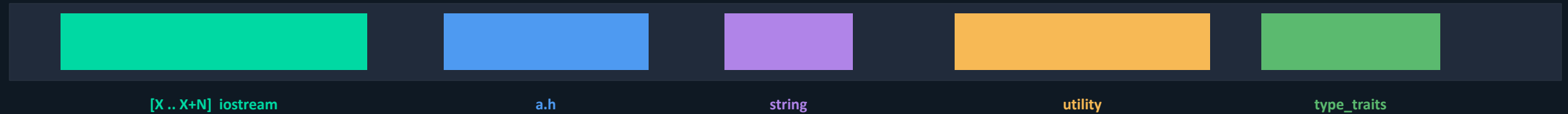
`DenseMap<FileEntryRef, (id, off)>`

- ✓ exact file identity match
- ✓ O(1) lookup
- ✗ one entry per file — no notion of ranges
- ✗ can't see overlap between two slabs

`llvm::IntervalMap<UIntTy, FileEntryRef>`

- ✓ keyed on global SLoc offset ranges
- ✓ O(log N) lookup, 1–2 cache lines profiled
- ✓ sees overlap, supports partial reuse
- ✓ matches address-space geometry

address space (loaded region) →

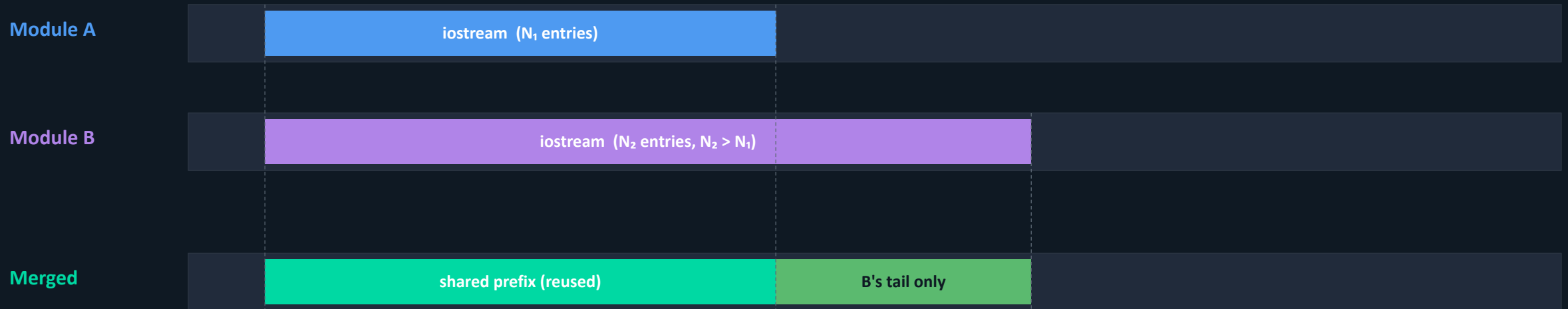


lookup: given a global offset, which interval contains it? → which file?

Why this shape? Because the address space *is* a sequence of intervals. *The data structure should look like the thing it describes.*

Two modules can serialize the same file differently

Same iostream. Different reader paths. *Module A serializes fewer SLocEntries than Module B for the same file — perhaps one transitively pulls in macro expansions the other doesn't.*



IntervalMap detects the overlap.

B reuses A's range for the shared prefix and allocates only the tail it actually adds. *DenseMap can't do this — equality is binary; overlap is structural.*

Adjust per-module mappings, not the world

The invariant ASTReader assumes

Each loaded module file `F` currently has **one contiguous** base offset. Every local SLoc offset in the .pcm is translated by `F.SLocEntryBaseOffset`.

```
// today, in ModuleFile:  
int      SLocEntryBaseID;  
UIntTy   SLocEntryBaseOffset;
```

Partial reuse breaks the single-base assumption — ranges in this file may map across two regions.

The contained fix — per-AST remap

Keep an **adjusted local mapping per AST file** inside ASTReader. When a local offset lands in a reused range, translate it via the merged base; otherwise via the file's own base.

```
// per ModuleFile, in ASTReader:  
SmallVector<LocalRemap>  
  SLocRemaps; // {local_range → global}
```

Scope stays inside ASTReader. SourceManager invariants untouched. Cleaner first; broader only if needed.

The hardest risk to predict without a prototype

⚠️ Sharing SLoc ranges between modules could cause a diagnostic from one module to resolve source text from another module's version of the same file.

Imagine:

Module A's view of a.h

```
#include <iostream>
int mod_A = 42;
```

diagnostic refers to line 2



expected: 'int mod_A = 42;'

...but range was shared with Module B's a.h

```
#include <iostream>
int mod_A = 999;    // differs!
```

resolver picks the wrong buffer



actual: 'int mod_A = 999;' x

"It's difficult to predict the full impact of these changes without a prototype. We recommend an explore-experiment-iterate approach."

— Vassil Vassilev, design note

Identity-gated dedup, with a regression suite

the dedup gate

```
// dedup only when files are verifiably identical:
if (Fcandidate.getInode() == Fexisting.getInode()
    && Fcandidate.getSize() == Fexisting.getSize())
    return ReuseExisting();          // safe path
else
    return AllocateFresh();          // fallback
```



Identity

FileEntryRef carries the inode. Two paths to the same file dedup correctly. Two same-named files with different content do not.



Falls back

Any mismatch — different inode, size, or unresolvable identity — returns to the original allocation path. Worst case is today's behavior.



Regression suite

Diagnostic regression suite runs after every meaningful change. LIT tests cover shared headers, symlinks, transitive imports, partial overlap.

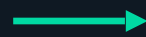
What the numbers should do

TWO-MODULE EXPERIMENT — TODAY

11,650

SLocEntries allocated

two modules, iostream allocated twice



AFTER DEDUP

5,827

SLocEntries allocated

iostream allocated once — back to baseline

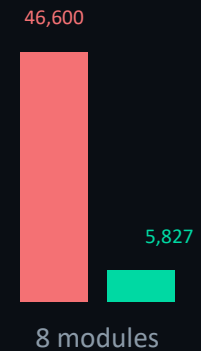
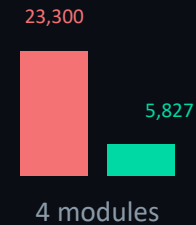
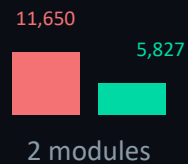
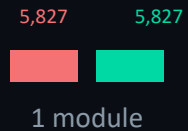
REDUCTION

50.0%

the duplication eliminated

N modules sharing the same headers → denominator stays flat

today after dedup



Five numbers that tell the whole story

PRIMARY

SLoc address space consumed

From -print-stats. Target: two-module case returns to 5,827. N modules → flat.

MEMORY

Peak resident set size

Measured with /usr/bin/time -v. Should drop proportionally to address space saved.

OVERHEAD

Wall-clock per compilation

Dedup mechanism itself adds no meaningful overhead. Any regression → bug, not cost.

LOOKUP COST

IntervalMap traversal

Profiled on real libc++ build. Target: 1–2 cache lines per traversal.

VISIBLE WIN

Redirected allocations counter

New counter in -print-stats. Directly observable measure of duplicates eliminated.

Timeline: May 4 — August 31



PHASE 1 · May 25 — Jul 5

DenseMap prototype

- Add LoadedFileEntryAllocations to SourceManager.h
- Dedup check inside AllocateLoadedSLocEntries
- Wire FileEntry identity through SOURCE_LOCATION_OFFSETS
- LIT tests: shared header, symlink, transitive import
- -print-stats counter for redirected allocations
- Full LLVM test suite → clean before midterm

Midterm success: two-module denominator → 5,827.

PHASE 2 · Jul 13 — Aug 23

IntervalMap integration

- Add IntervalMap::Allocator to SourceManager
- Replace DenseMap with IntervalMap (non-overlapping)
- Profile lookup on real libc++ — target 1–2 cache lines
- Confirm partial-overlap path with Vassil (week 9)
- Implement local remapping in ASTReader; new tests
- Edge cases: chained PCH, transitive reload paths

Final week: 3-patch upstream series on Discourse.

Stretch goals

1

UPSTREAM

Full upstream landing in 3 months

All three patches — dedup mechanism, test suite, IntervalMap integration — land before the fellowship ends. Includes responding to review iterations.

2

BROADER COVERAGE

Explicit module builds

Extend the experiment harness to cover `-fmodule-file=A=./A.pcm`. Vassil noted these consume even more resources than implicit builds but weren't investigated due to time.

3

ADJACENT WIN

Macro ID deduplication

The `-print-stats` output shows the same linear doubling for macros, types, identifiers. Investigate whether the dedup mechanism generalizes.

Ayokunle Amodu

BACKGROUND

Final-year Computing Science, University of Alberta

Research Assistant · Prof. J. Nelson Amaral, May–Aug 2024

Built a RISC-V dataflow analysis framework: CFG construction, dead code elimination, live variable analysis on non-SSA assembly. *Now part of the CMPUT 229 curriculum.*

TA · CMPUT 229, 200 students/semester

Daily exposure to 32-bit reasoning — what fits in an offset, what the hardware actually sees. Not abstract to me.

MLIR · Gazprea compiler, 4-person team

End-to-end codegen across the scf, memref, linalg, and llvm dialects.

LLVM CONTRIBUTIONS

Across ClangIR, MLIR, and Clang itself

Clang Diagnostics refactor to enum_select for better maintainability

ClangIR Atomic builtins: __sync_fetch_and_*, __builtin_isinf_sign, setjmp/longjmp, __builtin_bcopy, FP env RAI

ClangIR incubator Elementwise math builtins with ThroughMLIR lowerings

MLIR Crash fixes in the affine dialect

WHY I AM RIGHT FOR THIS

RISC-V is a 32-bit world where you reason constantly about **what fits where**. MLIR teaches you to trace meaning through **lowerings that must preserve invariants**. Both train you to sit with low-level invariants and be careful about assumptions — exactly what this work requires.

Thank you

Questions?

Optimize Usage of Source Locations in Clang Modules

